

A Report

on

GPU ALLOCATION AND MANAGEMENT SYSTEM

carried out as part of the course Project Based Learning - 2

Submitted by

Rohit Kumar Yadav

2427030335

IV Semester

in partial fulfillment for the award of the degree

of

BACHELOR OF TECHNOLOGY

In

Computer Science & Engineering



**MANIPAL UNIVERSITY
JAIPUR**

Department of Computer Science & Engineering,

School of Computer Science and Engineering,

Manipal University Jaipur,

Jan–May 2026

A Report

on

GPU ALLOCATION AND MANAGEMENT SYSTEM

carried out as part of the course Project Based Learning - 2

Submitted by

Rohit Kumar Yadav

2427030335

IV Semester

in partial fulfillment for the award of the degree

of

BACHELOR OF TECHNOLOGY

In

Computer Science & Engineering

Under the Guidance of :

Guide Name : Dr. Jay Shankar Sharma

Guide Signature (with date) :



**MANIPAL UNIVERSITY
JAIPUR**
(University under Section 2(f) of the UGC Act)

Department of Computer Science and Engineering

School of Computer Science and Engineering

Date: _____

CERTIFICATE

This is to certify that the project entitled "GPU Allocation and Management System" is a bonafide work carried out as PBL-2 End Term Assessment in partial fulfillment for the award of the degree of Bachelor of Technology in Computer Science and Engineering, by Rohit Kumar Yadav (Reg. No. 2427030335), during the academic semester IV of year 2025–2026.

Place: Manipal University Jaipur, Jaipur

Name of the project guide: _____

Signature of the project guide: _____

Acknowledgement

We express our sincere gratitude to our mentor, Dr. Jay Shankar Sharma, for his constant support, guidance, and continuous engagement throughout this project. We highly appreciate his technical comments, suggestions, and constructive criticism during the progress of the project titled “GPU Allocation and Management System.”

We owe our profound gratitude to the Head, Department of CSE, for her valuable guidance and for facilitating us during our work. We are also very grateful to all the faculty members and staff of the Department of Computer Science & Engineering, School of Computer Science and Engineering, Manipal University Jaipur, for their precious support and cooperation during the development of this project.

Finally, we extend our heartfelt appreciation to our classmates for their help, collaborative feedback, and encouragement throughout the development phases.

Rohit Kumar Yadav

2427030335

Abstract

The GPU Allocation and Management System is a centralized, web-based platform designed to address the critical problem of resource scarcity and inefficient scheduling in high-performance computing environments. In academic institutions such as Manipal University Jaipur, high-end GPU hardware is limited in availability and typically managed through informal, manual processes, which lead to scheduling conflicts, hardware underutilization, and a lack of transparency.

This project implements a full-stack MERN solution — built on MongoDB, Express.js, React.js, and Node.js — with Role-Based Access Control (RBAC) at its core. Three user roles are supported: Student, Faculty, and Administrator. Students and Faculty can browse available GPU nodes (e.g., NVIDIA RTX 4090, A100), submit allocation requests with VRAM requirements and project duration, and track approval status in real time. Administrators access a Global Command Center dashboard to monitor hardware health, manage the request queue, and ensure fair distribution of compute resources.

The frontend is built with React.js (Vite) and Tailwind CSS, delivering a modern responsive interface. Security is enforced through JSON Web Tokens (JWT) and Bcrypt password hashing. The system is containerized using Docker for deployment consistency.

Keywords: *GPU Management, MERN Stack, Resource Allocation, Role-Based Access Control, Web Application.*

Table of Contents

Acknowledgement	i
Abstract	ii
1. Introduction	1
1.1 Objective of the Project	1
1.2 Brief Description of the Project	2
1.3 Technology Used	2
1.3.1 Hardware Requirements	2
1.3.2 Software Requirements	3
2. Design Description	4
2.1 Flow Chart	4
2.2 Data Flow Diagrams (DFDs)	5
2.3 Entity Relationship Diagram (ER Diagram)	6
3. Project Description	7
3.1 Database	7
3.2 Table Description	7
3.3 File / Database Design	9
4. Input / Output Form Design	10
4.1 Input Design	10
4.2 Output Design	11
5. Testing & Tools Used	12
5.1 API Testing	12
5.2 Manual Testing	12

5.3 Browser Compatibility Testing	13
5.4 Database Validation	13
6. Implementation & Maintenance	14
6.1 Implementation	14
6.2 Security Implementation	14
6.3 Maintenance	15
7. Conclusion and Future Work	16
7.1 Conclusion	16
7.2 Future Work	17
8. Outcome	18
9. Bibliography	19

1. Introduction

The rapid growth of Artificial Intelligence, Deep Learning, and scientific computing has made Graphics Processing Units (GPUs) among the most valuable and sought-after computational resources in academic institutions. High-end GPUs such as the NVIDIA RTX 4090 and A100 are limited in number, expensive to procure, and require careful scheduling to ensure equitable access and maximum utilization.

At Manipal University Jaipur, the absence of a formal digital system for GPU resource management has historically led to scheduling conflicts, idle hardware during off-peak hours, and a lack of accountability in usage. This project addresses these challenges by providing a structured, role-based web application to manage the full lifecycle of GPU allocation — from initial request submission to administrative approval and hardware release.

1.1 Objective of the Project

The primary objectives of the GPU Allocation and Management System are:

- **Automated Scheduling:** Eliminate manual, spreadsheet-based tracking of GPU usage and replace it with a digital request-and-approval workflow.
- **Resource Monitoring:** Provide real-time visibility into the operational status (Available, In Use, Maintenance, Allocated) of all registered hardware nodes.
- **Fair Access Control:** Implement a transparent approval workflow where GPU allocation requests are formally vetted and authorized by administrators or faculty.
- **Conflict Prevention:** Use database-level constraints and status management to ensure no two users are allocated the same hardware resource simultaneously.
- **Scalability:** Design the system in a modular fashion that can easily accommodate new GPU nodes, user roles, or institutional policies.

1.2 Brief Description of the Project

The system functions as a tiered web portal, with each user role accessing a customized dashboard tailored to their responsibilities. Students and Faculty can log in to browse the catalogue of registered GPU nodes, view each unit's current availability, and submit an allocation request by specifying their project purpose, required VRAM, and intended usage

dates. Once submitted, the request transitions to a PENDING state that the user can track in real time.

Administrators access a Global Command Center featuring visual StatCards that display system-wide usage statistics, hardware availability, and pending request counts. From this panel, they can add new hardware nodes, approve or reject requests, update hardware status, and review historical usage logs.

1.3 Technology Used

1.3.1 Hardware Requirements

Component	Minimum Specification
Processor	Intel Core i5 (8th Gen or higher) / AMD equivalent
RAM	8 GB (16 GB recommended for development)
Storage	20 GB free disk space
Network	Stable internet connection for API and DB access
Display	1280 × 720 resolution or higher

1.3.2 Software Requirements

Layer / Category	Technology / Tool
Frontend Framework	React.js (Vite build tool), ShadCN-inspired component library
Styling	Tailwind CSS — responsive, utility-first design
Backend Framework	Node.js with Express.js (REST API architecture)
Database	MongoDB (NoSQL) with Mongoose ODM
Authentication	JSON Web Tokens (JWT) for stateless session management
Password Security	Bcrypt for password hashing and salting
API Testing	Postman for endpoint validation
Containerization	Docker for environment consistency and deployment

2. Design Description

This chapter presents the architectural and logical design of the GPU Allocation and Management System. The design encompasses the application flow, data movement across layers, and entity relationships that collectively define the system's structure.

2.1 Flow Chart

The application logic is governed by four sequential processing layers:

Layer 1 – Authentication Layer

- The user enters their registered credentials (username and password) on the login page.
- The system validates the credentials against the database. On success, a signed JWT token is generated and stored on the client side.
- The user's role is read from the token payload and they are redirected to the appropriate role-specific dashboard (Student / Faculty / Admin).

Layer 2 – Request Submission Layer

- An authenticated student or faculty member selects an available GPU node from the catalogue.
- The user completes a request form specifying the project purpose, required VRAM, start date, and end date.
- Upon submission, a new GPU Request document is created in the database with status PENDING.

Layer 3 – Administrative Approval Layer

- The administrator views the Pending Requests table on the admin dashboard.
- The administrator reviews the submitted details and clicks Approve or Reject.
- On approval, the request status updates to APPROVED and the GPU resource status updates to Allocated. On rejection, the status is set to REJECTED.

Layer 4 – Hardware Status Layer

- Available → Allocated (upon request approval)

- Allocated → Available (upon project completion or manual release)
- Available → Maintenance (when flagged by an administrator for servicing)

2.2 Data Flow Diagrams (DFDs)

Level 0 – Context Diagram

At the highest level, the system interacts with one external entity: the User. The User sends authentication credentials and receives a role-based access token in return. The User can then submit a Resource Request and receive Status Updates. The Administrator, as a specialized user, additionally interacts with the system to manage hardware and process the approval queue.

Level 1 – Detailed DFD

- Process 1.0 – User Authentication: Receives credentials, queries the User Data Store, validates the password hash via Bcrypt, and emits a JWT token to the client.
- Process 2.0 – Request Processing: Receives a validated allocation request from an authenticated user, creates a PENDING record in the GPU Request Data Store, and notifies the admin queue.
- Process 3.0 – Admin Approval Engine: Reads pending requests. Upon admin action, updates request status and writes the new GPU resource status (Allocated) to the GPU Resource Data Store.
- Process 4.0 – Hardware Management: Allows the administrator to create, update, or deactivate GPU Resource records, which are reflected immediately in the user-facing catalogue.

2.3 Entity Relationship Diagram (ER Diagram)

Entities and Attributes

Entity	Attribute	Description
USER	username	Unique string identifier for the user account
USER	role	Enum: STUDENT, FACULTY, or ADMIN

USER	password	Bcrypt-hashed password (never stored in plaintext)
GPU_RESOURCE	name	Human-readable label (e.g., RTX 4090 Lab 1)
GPU_RESOURCE	model	Hardware model name (e.g., NVIDIA RTX 4090)
GPU_RESOURCE	vram	Video RAM capacity in gigabytes (numeric)
GPU_RESOURCE	status	Enum: Available, In Use, Maintenance, Allocated
GPU_REQUEST	userId	Foreign key reference to USER entity
GPU_REQUEST	gpuResourceId	Foreign key reference to GPU_RESOURCE entity
GPU_REQUEST	purpose	Text description of the project use-case
GPU_REQUEST	startDate / endDate	Date range for the requested allocation period
GPU_REQUEST	status	Enum: PENDING, APPROVED, REJECTED, COMPLETED

Relationships

- One USER can submit many GPU_REQUESTs (One-to-Many).
- One GPU_RESOURCE can be referenced by many GPU_REQUESTs, but only one APPROVED request may be active per resource at any time (enforced via application logic).
- GPU_REQUEST acts as an associative entity linking USER and GPU_RESOURCE.

3. Project Description

3.1 Database

The system uses MongoDB, a document-oriented NoSQL database, to persist all application data. MongoDB stores data as flexible BSON (Binary JSON) documents, making it well-suited for managing heterogeneous GPU hardware configurations and dynamic, user-generated request logs. The Mongoose Object Data Modeling (ODM) library is used on

top of MongoDB to enforce schema structure, data validation rules, and pre-save hooks within the Node.js application layer.

The database consists of three primary collections: users, gpuresources, and gpurequests. These collections correspond directly to the three Mongoose schemas described in Section 3.2 below.

3.2 Table Description

A. User Schema

The User schema manages all registered accounts within the system. Each user document is uniquely identified by their username, and the role field controls access permissions throughout the application.

Field Name	Data Type	Description
username	String (Unique)	Unique identifier for the user account (indexed)
role	Enum	One of: STUDENT, FACULTY, ADMIN
password	String (Hashed)	Bcrypt-hashed password — never stored in plaintext

B. GPU Resource Schema

The GPU Resource schema represents each physical GPU node registered in the system. The status field is the primary indicator used to determine availability for allocation.

Field Name	Data Type	Description
name	String	Descriptive label for the node (e.g., RTX 4090 Lab 1)
model	String	Hardware model name (e.g., NVIDIA RTX 4090)
vram	Number	Video RAM capacity in gigabytes
status	Enum	One of: Available, In Use, Maintenance, Allocated

C. GPU Request Schema

The GPU Request schema acts as the central transactional record of the system. It captures a user's request for a specific GPU node, the intended usage window, and the administrative decision rendered against it.

Field Name	Data Type	Description
userId	ObjectId (Ref: User)	Reference to the User document who submitted the request
gpuResourceId	ObjectId (Ref: GpuResource)	Reference to the target GPU Resource document
purpose	String	Description of the project or task requiring GPU access
startDate	Date	Intended start date for the GPU allocation
endDate	Date	Intended end date; must be after startDate (pre-save validated)
status	Enum	One of: PENDING, APPROVED, REJECTED, COMPLETED

3.3 File / Database Design

The backend follows the MVC (Model-View-Controller) architectural pattern, organized into the following layers:

- **Models:** Mongoose schemas defining the data structure and validation rules for each collection (User, GpuResource, GpuRequest).
- **Routes:** Dedicated Express.js route files for authentication (/api/auth), GPU resource management (/api/gpus), and request management (/api/requests).
- **Controllers:** Business logic handlers that process incoming HTTP requests, interact with the database via Mongoose, and return structured JSON responses.
- **Middleware:** JWT verification middleware that intercepts protected routes to validate the token and attach the decoded user payload (including role) to the request object. A separate RBAC middleware layer checks the user's role before granting access to administrator-only endpoints.

4. Input / Output Form Design

4.1 Input Design

Login Form

The login screen presents a clean form accepting the user's registered username and password. Client-side validation ensures both fields are populated before submission. Server-side error messages are displayed inline for invalid credentials without disclosing which field is incorrect (a security best practice).

GPU Request Modal

Authenticated students and faculty access a modal form triggered from the GPU catalogue.

This form requires the following inputs:

- GPU Node Selection: Pre-populated with the selected hardware node's name and model.
- Project Purpose: A free-text field describing the intended use of the GPU.
- Start Date and End Date: Date pickers constrained to prevent past-date selection and to ensure the end date is after the start date. Validated on both client and server (Mongoose pre-save hook).

Admin Hardware Form

Administrators can register a new GPU node through a dedicated form, providing the node name, hardware model, and VRAM capacity. Upon submission, the node is added to the database with an initial status of Available.

4.2 Output Design

Admin Dashboard

The administrative interface prominently features visual StatCards displaying aggregate metrics such as total registered GPUs, available nodes, pending requests, and total active users. Below the StatCards, a structured table lists all pending requests with user details, the requested GPU, and action buttons for Approve and Reject.

Student / Faculty Dashboard

The user-facing dashboard displays a catalogue of GPU nodes with their current availability status. Submitted requests appear in a personal request history table, each row showing the GPU name, submission date, and a color-coded status badge.

Status Indicators

Color	Status Represented
Green	Available / Approved / Completed
Orange / Amber	Pending / Maintenance
Red	Rejected / In Use
Blue / Grey	Allocated

5. Testing & Tools Used

5.1 API Testing

All backend REST API endpoints were tested comprehensively using Postman. Tests covered each of the CRUD operations for GPU resources and GPU requests:

- Authentication Endpoints: POST /api/auth/register and POST /api/auth/login were tested for valid credential acceptance, duplicate registration prevention, and incorrect-password rejection.
- GPU Resource Endpoints: GET /api/gpus, POST /api/gpus, PUT /api/gpus/:id, and DELETE /api/gpus/:id were verified for correct data persistence and appropriate HTTP response codes.
- GPU Request Endpoints: POST /api/requests, GET /api/requests, and PATCH /api/requests/:id/status were tested for role-based access enforcement and correct status transitions.

5.2 Manual Testing

End-to-end user journeys were simulated manually to validate the complete application flow:

1. New user registration followed by role-based login and redirection.
2. Student submits a GPU request; admin sees it as PENDING.

3. Admin approves the request; GPU status changes to Allocated; student sees APPROVED.
4. Admin rejects a request; student sees REJECTED badge.
5. Admin marks a GPU node as Maintenance; it becomes unavailable for new requests.

5.3 Browser Compatibility Testing

The frontend was tested on Google Chrome (latest stable) and Mozilla Firefox (latest stable) to verify consistent rendering of Tailwind CSS utility classes, responsive grid layouts, and modal components. No major rendering discrepancies were identified.

5.4 Database Validation

Mongoose pre-save hooks were validated to enforce the following schema-level constraints:

- **Date Validation:** The `endDate` field on GPU requests correctly rejects documents where the end date is equal to or prior to the start date.
- **Enum Validation:** Invalid values for role, GPU status, and request status fields are rejected by Mongoose before database insertion.
- **Uniqueness Constraint:** Duplicate username registration attempts correctly return an error from the unique index on the User schema.

6. Implementation & Maintenance

6.1 Implementation

The complete application stack — comprising the React frontend, Node.js/Express backend, and MongoDB instance — is containerized using Docker. A `docker-compose.yml` configuration file orchestrates the simultaneous startup of all three services, manages inter-container networking, and maps the necessary ports to the host machine. This ensures consistent deployment across different developer machines and server environments without dependency conflicts.

6.2 Security Implementation

- **Authentication:** All users must authenticate via the login endpoint, which returns a signed JWT token upon success. This token must be included in the Authorization header of all subsequent requests to protected endpoints.
- **Authorization (RBAC):** A middleware function on the server decodes the JWT token and verifies the user's role before permitting access to sensitive routes. Students cannot access administrative routes.
- **Password Security:** User passwords are salted and hashed using Bcrypt (cost factor 10) before being persisted to the database. Passwords are never stored in plaintext.

6.3 Maintenance

- **Hardware Deactivation:** Administrators can update any GPU node's status to Maintenance, immediately removing it from the list of resources available for new student requests.
- **Request Audit:** All historical GPU requests (approved, rejected, and completed) are retained in the database, providing a complete audit trail for accountability and usage pattern analysis.
- **User Management:** The system is architected to support future extensions such as account deactivation or role reassignment.

7. Conclusion and Future Work

7.1 Conclusion

The GPU Allocation and Management System successfully delivers a robust, scalable, and secure solution for managing high-value computational assets in an academic setting. The project effectively addresses the core problem of unstructured, conflict-prone GPU scheduling by providing a transparent, role-based, and auditable digital workflow.

The system fulfills all stated objectives: it eliminates manual tracking, provides real-time hardware status visibility, enforces fair access through an administrative approval process, and prevents allocation conflicts through database-level status management. The MERN stack architecture proved to be an effective choice, enabling rapid full-stack development while maintaining clean separation of concerns.

7.2 Future Work

- **Auto-Scheduling with Queue Management:** Implementation of a priority queue for high-demand GPU nodes, enabling the system to automatically schedule the next allocation upon completion of the current one.
- **Real-Time GPU Metrics Integration:** Integration with NVIDIA System Management Interface (nvidia-smi) to display live telemetry data — including GPU utilization, memory usage, and temperature — on the admin dashboard.
- **Email / Push Notifications:** Automated email or in-app notifications to inform users when their request has been approved, rejected, or when their allocation period is approaching its end date.
- **Cloud Bursting:** Development of a module to provision cloud-based GPU instances (e.g., AWS EC2 P-instances or Google Cloud TPUs) when all local hardware nodes are at capacity.
- **Analytics Dashboard:** A dedicated analytics panel providing visual reports on GPU utilization trends, peak demand periods, and user-wise consumption statistics to assist administrators in capacity planning.

8. Outcome

The primary outcome of this project is a functional, production-ready web application that automates the complete lifecycle of GPU resource allocation. The system transitions Manipal University Jaipur's GPU management from a manual, error-prone process to a structured, transparent digital workflow.

Specifically, the project delivers the following outcomes:

- A fully operational, role-based web application accessible to Students, Faculty, and Administrators.
- A secure authentication system utilizing JWT and Bcrypt, ensuring resources are accessible only to authorized personnel.
- A real-time administrative dashboard providing complete visibility into hardware availability and request queues.
- A containerized deployment package (Docker) enabling consistent and reproducible deployment.
- A clean, well-documented RESTful API ready for future integration with external systems or mobile clients.

Progress towards outcome: The web application is fully developed and operational on a local Docker-based deployment. API documentation is prepared, and the system is ready for institutional pilot testing. A technical report documenting the implementation details has been prepared as part of this submission.

9. Bibliography

- [1] Meta Open Source. React Documentation — Component Lifecycle and State Management. Available at: <https://react.dev/> (Accessed: April 2026).
- [2] OpenJS Foundation. Express.js Documentation — Middleware and Routing Strategies. Available at: <https://expressjs.com/> (Accessed: April 2026).
- [3] MongoDB, Inc. MongoDB Documentation — Database Design for High-Concurrency Resources. Available at: <https://www.mongodb.com/docs/> (Accessed: April 2026).
- [4] Automattic. Mongoose ODM Documentation — Schema Validation and Pre-save Hooks. Available at: <https://mongoosejs.com/docs/> (Accessed: April 2026).
- [5] Auth0. JSON Web Tokens (JWT) — Introduction to JWT for Stateless Authentication. Available at: <https://jwt.io/introduction/> (Accessed: April 2026).
- [6] npm Registry. bcrypt.js — Password Hashing for Node.js. Available at: <https://www.npmjs.com/package/bcrypt> (Accessed: April 2026).
- [7] Tailwind Labs. Tailwind CSS Documentation — Utility-First CSS Framework. Available at: <https://tailwindcss.com/docs/> (Accessed: April 2026).
- [8] Docker, Inc. Docker Documentation — Containerization for Consistent Deployment Environments. Available at: <https://docs.docker.com/> (Accessed: April 2026).
- [9] Evan You. Vite Documentation — Next Generation Frontend Tooling. Available at: <https://vitejs.dev/> (Accessed: April 2026).
- [10] NVIDIA Corporation. NVIDIA System Management Interface (nvidia-smi). Available at: <https://developer.nvidia.com/nvidia-system-management-interface> (Accessed: April 2026).